IFA Preload DD Kernel 模板完整代码解析

> **对应PDF**: 06_IFA_Preload_DD_Kernel_Full_Analysis.pdf
> **源文件**: incre_flash_attention_preload_dd.h (3991行)

一、Tiling 切分逻辑详解

1.1 Tiling 数据结构定义

代码位置: `op_host/incre_flash_attention_tiling.h:44-151`

```
// Tiling核心参数结构体
BEGIN_TILING_DATA_DEF(IncreFlashAttentionBaseParams)
TILING_DATA_FIELD_DEF(uint32_t, batchSize)      // Batch数量
TILING_DATA_FIELD_DEF(uint32_t, seqSize)        // KV序列长度
TILING_DATA_FIELD_DEF(uint32_t, qSeqSize)       // Q序列长度
TILING_DATA_FIELD_DEF(uint32_t, headSize)       // Head维度
TILING_DATA_FIELD_DEF(uint32_t, kvHeadNum)      // KV Head数量
TILING_DATA_FIELD_DEF(uint32_t, qHeadNum)       // Q Head数量
TILING_DATA_FIELD_DEF(uint32_t, nNumOfQInOneGroup) // GQA分组大小
END_TILING_DATA_DEF

BEGIN_TILING_DATA_DEF(IncreFlashAttentionSingleCoreParams)
TILING_DATA_FIELD_DEF(uint32_t, sInnerLoopTimes);      // S内循环次数
TILING_DATA_FIELD_DEF(uint32_t, singleProcessSInnerSize); // 单次处理S大小
TILING_DATA_FIELD_DEF(uint32_t, singleProcessSInnerSizeTail); // 尾部S大小
TILING_DATA_FIELD_DEF(uint32_t, usedCoreNum);          // 使用核数
TILING_DATA_FIELD_DEF(uint32_t, formerCoreNum);        // 前置核数
TILING_DATA_FIELD_DEF(uint32_t, groupSplitSize);       // G方向切分大小
TILING_DATA_FIELD_DEF(uint32_t, s1SplitSize);          // S1方向切分大小
END_TILING_DATA_DEF
```

1.2 Kernel侧Tiling参数读取

代码位置: `incre_flash_attention_preload_dd.h:585-638`

```
template <typename IFAT> __aicore__ inline void IncreFlashAttentionAttenPreloadDD<IFAT>::InitTilingData()
{
    // ========== 核心Tiling参数读取 ==========

     // 1. S方向切分参数
     singleProcessSInnerSize = tilingData->increFlashAttentionSingleCoreParams.singleProcessSInnerSize;
    // 含义: 单次处理的KV序列长度，典型值1024/2048/8192
    // 影响: 决定每次BMM1计算的N维度大小

     sInnerLoopTimes = tilingData->increFlashAttentionSingleCoreParams.sInnerLoopTimes;
    // 含义: S方向需要循环多少次
    // 计算: sInnerLoopTimes = ceil(kvSeqLen / singleProcessSInnerSize)

     singleProcessSInnerSizeTail = tilingData->increFlashAttentionSingleCoreParams.singleProcessSInnerSizeTail;
    // 含义: 最后一次循环的S大小（可能小于singleProcessSInnerSize）

    // 2. 核数分配参数
     usedCoreNum = tilingData->increFlashAttentionSingleCoreParams.usedCoreNum;
     formerCoreNum = tilingData->increFlashAttentionSingleCoreParams.formerCoreNum;
    // 含义: 前者处理较多任务，后者处理较少任务

     // 3. Batch/Head参数
     batchSize = tilingData->baseParams.batchSize;
     kvHeadNum = tilingData->baseParams.kvHeadNum;
     qHeadNum = tilingData->baseParams.qHeadNum;
     gSize = tilingData->baseParams.nNumOfQInOneGroup;  // GQA: 每组Query数量

    // 4. 维度参数
     headDim = tilingData->baseParams.headSize;
     headDimAlign = Align(headDim, BYTE_BLOCK);  // 对齐到32字节
     // headDim=128时, headDimAlign=128
     // headDim=96时, headDimAlign=96 (INT8) 或128 (FP16)

     // 5. G/S1切分参数
     gSizeSub = tilingData->increFlashAttentionSingleCoreParams.groupSplitSize;
     gOuter = (gSize + gSizeSub - 1) / gSizeSub;  // G方向切分次数
     gSizeTail = gSize - (gOuter - 1) * gSizeSub; // G尾部大小

     s1SizeSub = tilingData->increFlashAttentionSingleCoreParams.s1SplitSize;
     s1Outer = (qSeqSize + s1SizeSub - 1) / s1SizeSub;  // S1方向切分次数
     s1SizeTail = qSeqSize - (s1Outer - 1) * s1SizeSub; // S1尾部大小

     // 6. MSD迭代次数
     msdIterNum = tilingData->baseParams.msdIterNum;
```

```
    // 含义：MSD双段反量化迭代次数，伪量化场景=2
}
```

1.3 S方向动态切分计算

代码位置: `incre_flash_attention_preload_dd.h:815-843`

```
template <typename IFAT> __aicore__ inline void IncreFlashAttentionAttenPreloadDD<IFAT>::UpdateInnerLoopCond()
{
    // ========== 动态更新S方向循环条件 ==========

    if (curActualSeqLen == 0) {
        curActSeqLenIsZero = true;
        return;  // 序列长度为0，跳过处理
    }
    curActSeqLenIsZero = false;

    int32_t remainSinnerSize = (int32_t)curActualSeqLen;
    int32_t computeSinnerSize = (int32_t)curActualSeqLen;

    // Flash Decode场景：根据s2Idx计算当前块
    if constexpr (FLASH_DECODE) {
        remainSinnerSize = (int32_t)curActualSeqLen - sInnerLoopSize * s2IdxFD;
        computeSinnerSize = remainSinnerSize >= sInnerLoopSize ? sInnerLoopSize : remainSinnerSize;
    }

    // ========== 计算循环次数和尾部大小 ==========
    if (remainSinnerSize > 0) {
        if (computeSinnerSize <= singleProcessSInnerSize) {
          // 单次处理即可完成
            singleProcessSInnerSizeTail = computeSinnerSize;
            sInnerLoopTimes = 1;
        } else {
          // 需要多次循环
            sInnerLoopTimes = (computeSinnerSize + singleProcessSInnerSize - 1) / singleProcessSInnerSize;
            singleProcessSInnerSizeTail = computeSinnerSize - (sInnerLoopTimes - 1) * singleProcessSInnerSize;
       }
    } else {
        sInnerLoopTimes = 0;
    }
}
```

关键逻辑解析:
- `sInnerLoopTimes`： 根据实际KV序列长度动态计算
- `singleProcessSInnerSizeTail`： 尾部可能不完整，需要特殊处理

二、Workspace 计算逻辑详解

2.1 Workspace组成部分

代码位置: `incre_flash_attention_preload_dd.h:947-999`

```
template <typename IFAT> __aicore__ inline void IncreFlashAttentionAttenPreloadDD<IFAT>::Init(...)
{
    uint64_t offset = 0;

    // ========== Workspace分配顺序 ==========

    // 1. Q预量化结果（伪量化场景）
    if constexpr (ANTIQUANT) {
        if constexpr (KVINT4) {
            // INT4场景：数据量减半
            queryPreProcessResGm.SetGlobalBuffer(
                (__gm__ KV_T *)(workspace + offset +
                    (aiCoreIdx * dbWorkspaceRatio * bmm2ResUbSize * sizeof(KV_T) >> 1)));
            offset += (GetBlockNum() * dbWorkspaceRatio * bmm2ResUbSize * sizeof(KV_T) >> 1);
        } else {
            queryPreProcessResGm.SetGlobalBuffer(
                (__gm__ KV_T *)(workspace + offset +
                    aiCoreIdx * dbWorkspaceRatio * bmm2ResUbSize * sizeof(KV_T)));
            offset += GetBlockNum() * dbWorkspaceRatio * bmm2ResUbSize * sizeof(KV_T);
       }
    }

    // 2. MM1结果（BMM1输出）
    mm1ResGm.SetGlobalBuffer(
        (__gm__ MM_OUT_T *)(workspace + offset +
            aiCoreIdx * dbWorkspaceRatio * mmResUbSize * sizeof(MM_OUT_T)));
    offset += GetBlockNum() * dbWorkspaceRatio * mmResUbSize * sizeof(MM_OUT_T);

    // 3. Vec1结果（Softmax输出）
    vec1ResGm.SetGlobalBuffer(
        (__gm__ KV_T *)(workspace + offset +
```

```
            aiCoreIdx * dbWorkspaceRatio * mmResUbSize * sizeof(KV_T)));
    offset += GetBlockNum() * dbWorkspaceRatio * mmResUbSize * sizeof(KV_T);

    // 4. MM2结果（BMM2输出）
    mm2ResGm.SetGlobalBuffer(
        (__gm__ MM_OUT_T *)(workspace + offset +
            aiCoreIdx * dbWorkspaceRatio * bmm2ResUbSize * sizeof(MM_OUT_T)));
    offset += GetBlockNum() * dbWorkspaceRatio * bmm2ResUbSize * sizeof(MM_OUT_T);

    // 5. Vec2结果（累积输出）
    vec2ResGm.SetGlobalBuffer(
        (__gm__ MM_OUT_T *)(workspace + offset +
            aiCoreIdx * dbWorkspaceRatio * bmm2ResUbSize * sizeof(MM_OUT_T)));
    offset += GetBlockNum() * dbWorkspaceRatio * bmm2ResUbSize * sizeof(MM_OUT_T);
}
```

2.2 Workspace大小计算公式

Host侧Tiling计算（参考 `op_host/incre_flash_attention_tiling_impl.h`）：

```
// Workspace计算公式（典型配置）
// dbWorkspaceRatio = PRE_LOAD_NUM_DD = 4

// mmResUbSize = mSize * sInnerSizeAlign * msdIterNum
// mSize = 64（典型值）
// sInnerSizeAlign = 2048（GQA+DD场景）
// msdIterNum = 2（伪量化场景）
// => mmResUbSize = 64 * 2048 * 2 = 262144元素

// bmm2ResUbSize = mSize * headDimAlign * msdIterNum
// => bmm2ResUbSize = 64 * 128 * 2 = 16384元素

// Workspace总大小：
// mm1ResGm: 24核 × 4槽 × 262144 × 2B(FP16) = ~50MB
// vec1ResGm: 24核 × 4槽 × 262144 × 1B(INT8) = ~25MB
// mm2ResGm: 24核 × 4槽 × 16384 × 2B = ~3MB
// vec2ResGm: 24核 × 4槽 × 16384 × 4B(FP32) = ~6MB
// 总计：~84MB（不含Flash Decode额外空间）
```

2.3 dbWorkspaceRatio的关键作用

代码位置: `incre_flash_attention_preload_dd.h:328`

```
const uint32_t dbWorkspaceRatio = PRE_LOAD_NUM_DD;  // = 4
```

含义解析:
- `dbWorkspaceRatio`：Workspace预加载槽数
- 每个AICore分配 `dbWorkspaceRatio` 个槽位
- 槽位轮转：`(info.loop % PRE_LOAD_NUM_DD)` 确定当前使用哪个槽

轮转索引示例:

```
// 代码位置：incre_flash_attention_preload_dd.h:2230
uint64_t inOutGmOffset = (info.loop % PRE_LOAD_NUM_DD) * mmResUbSize + ...;
// loop=0 -> slot 0
// loop=1 -> slot 1
// loop=2 -> slot 2
// loop=3 -> slot 3
// loop=4 -> slot 0（重新轮转）
```

三、关键参数计算逻辑详解

3.1 PRE_LOAD_NUM_DD vs TASK_NUM

代码位置: `incre_flash_attention_preload_dd.h:35-39`

```
#define PRE_LOAD_NUM_DD 4    // 预加载Buffer槽数量
#define EXTRA_NUM 6          // ExtraInfo数组大小
#define TASK_NUM 2           // 任务并行深度
```

三参数区别:

参数	值	定义位置	用途	影响
PRE_LOAD_NUM_DD	4	:35	Workspace轮转槽数	决定中间结果存储深度
TASK_NUM	2	:38	任务派发深度	决定同时派发任务数

参数	值	定义位置	用途	影响
EXTRA_NUM	6	:37	状态数组大小	保存中间计算状态

使用场景:

```
// PRE_LOAD_NUM_DD：用于workspace索引
uint64_t offset = (info.loop % PRE_LOAD_NUM_DD) * size;

// TASK_NUM：用于任务并行派发
TaskContext taskContext[TASK_NUM];  // 同时处理2个任务

// EXTRA_NUM：用于ExtraInfo数组
ExtraInfo extraInfo[EXTRA_NUM];      // 保存6轮中间状态
```

3.2 MSD双段反量化系数

代码位置: `incre_flash_attention_preload_dd.h:256-259`

```
static constexpr float quantScaleC1S1 = 1.0 / (1024);              // MSD高段系数
static constexpr float quantScaleC1S2 = 1.0 / (1024 * 254);       // MSD低段系数
static constexpr float quantScaleC2O1 = 1.0 / (1024 * 127);       // MM2高段系数
static constexpr float quantScaleC2O2 = 1.0 / (1024 * 254 * 127); // MM2低段系数
```

数学原理:

```
MSD双段分解:
V_int8 = V_MSD × 254 + V_LSB

反量化公式:
V_fp16 = V_MSD × quantScaleC1S1 + V_LSB × quantScaleC1S2
       = V_MSD × (1/1024) + V_LSB × (1/(1024×254))

精度分析:
- 有效量化级别: 254 × 254 ≈ 65024
- 理论精度: log2(65024) ≈ 16bit
- 实际精度: 受FP16尾数10bit限制，约14bit

为什么是254而非256?
- 避免第二段饱和溢出
- INT8范围[-128,127]，正向最大127，负向最小-128
- 留2个量化级别的margin
```

3.3 Buffer容量参数

代码位置: `incre_flash_attention_preload_dd.h:262-269`

```
// L1 Buffer容量
static constexpr uint32_t L1Q_BLOCK_SIZE = 64 * 1024;    // Q: 64KB
static constexpr uint32_t L1KP_BLOCK_SIZE = 64 * 1024;   // K/P: 64KB
static constexpr uint32_t L1V_BLOCK_SIZE = 64 * 1024;    // V: 64KB

// L0 Buffer容量
static constexpr uint32_t L0A_PP_SIZE = (32 * 1024);     // L0A: 32KB
static constexpr uint32_t L0B_PP_SIZE = (32 * 1024);     // L0B: 32KB
static constexpr uint32_t L0C_PP_SIZE = (64 * 1024);     // L0C: 64KB
```

L1分配策略:

```
// 代码位置: incre_flash_attention_preload_dd.h:676-681
pipe->InitBuffer(tmpBufL1Q, L1Q_BLOCK_SIZE);                        // 64KB × 1 = 64KB
pipe->InitBuffer(tmpBufL1KP, 3 * L1KP_BLOCK_SIZE);                  // 64KB × 3 = 192KB
pipe->InitBuffer(tmpBufL1V, 4 * L1V_BLOCK_SIZE);                    // 64KB × 4 = 256KB
// 总计: 64KB + 192KB + 256KB = 512KB < 1MB L1容量
```

为什么是3×KP和4×V:
- KP需要3槽：BMM1的K矩阵 + BMM2的P矩阵交替使用
- V需要4槽：与PRE_LOAD_NUM_DD=4对应，深度预加载

四、核心函数逐行解析

4.1 Process 主流程函数

代码位置: `incre_flash_attention_preload_dd.h:3779-3954`

```
template <typename IFAT>
__aicore__ inline void IncreFlashAttentionAttenPreloadDD<IFAT>::Process()
{
```

```
    // ========= 1. 初始化同步Flags ==========

     if ASCEND_IS_AIV {
         // Vector侧: 设置输入输出Buffer同步Flags
         SetFlag<AscendC::HardEvent::V_MTE2>(SYNC_INPUT_BUF1_FLAG);
         SetFlag<AscendC::HardEvent::V_MTE2>(SYNC_INPUT_BUF2_FLAG);
         SetFlag<AscendC::HardEvent::MTE3_V>(SYNC_OUTPUT_BUF1_FLAG);
         SetFlag<AscendC::HardEvent::MTE3_V>(SYNC_OUTPUT_BUF2_FLAG);
    }

    // ========= 2. 核数判断 ==========

     if (aiCoreIdx < usedCoreNum) {
        // 只处理分配到当前核的任务

        // ========= 3. 初始化Softmax默认值 ==========

         if ASCEND_IS_AIV {
             // Vector侧: 初始化Softmax max/sum默认值
#ifdef IFA_SOFTMAX_WITHOUT_BRC_PRELOAD
             Duplicate(softmaxMaxDefaultUb, SOFTMAX_MIN_NUM, SOFTMAX_MAX_BUF_SIZE);
             Duplicate(softmaxSumDefaultUb, FLOAT_ZERO, SOFTMAX_SUM_BUF_SIZE);
#else
             Duplicate(softmaxMaxDefaultUb, SOFTMAX_MIN_NUM, BUFFER_SIZE_BYTE_512B / sizeof(T));
             Duplicate(softmaxSumDefaultUb, FLOAT_ZERO, BUFFER_SIZE_BYTE_512B / sizeof(T));
#endif
         } else {
             // Cube侧: 设置L1/L0同步Flags初始状态
             SetFlag<HardEvent::MTE1_MTE2>(L1Q_EVENT0);
             SetFlag<HardEvent::MTE1_MTE2>(L1KP_EVENT0);
           // ... 其他Flags
        }

        // ========= 4. 初始化ExtraInfo和TaskContext ==========

         ExtraInfo extraInfo[EXTRA_NUM];      // 中间状态数组
         TaskContext taskContext[TASK_NUM];   // 任务上下文数组

         uint32_t gloop = 0;       // 全局循环计数
         uint32_t gtasks = 0;      // 全局已处理任务数
         uint32_t gtasksC1V1 = 0;  // C1-V1阶段已处理任务数
         uint32_t gtasksC2V2 = 0;  // C2-V2阶段已处理任务数
         uint32_t tasks = 0;       // 当前待派发任务数

        // ========= 5. 跳过空序列任务 ==========

         if constexpr (!FLASH_DECODE) {
             for (uint32_t bn2gIdx = bn2LoopTimes; bn2gIdx > 0; bn2gIdx--) {
                 GetBN2Gid(bn2gIdx - 1);
                 GetActualSeqLen(bIdx, s1Idx);
                 if (curActualSeqLen == 0) {
                     DealActSeqLenIsZero(bIdx, n2Idx);  // 输出置零
              }
                 if (curActualSeqLen != 0 && actS1Size != 0) {
                     break;  // 找到有效任务
              }
                 bn2LoopTimes--;  // 减少总循环次数
           }
        }

        // ========= 6. 主循环: 遍历Batch/N2/S1 ==========

         for (uint32_t bn2gIdx = 0; bn2gIdx < bn2LoopTimes; bn2gIdx++) {
             GetBN2Gid(bn2gIdx);           // 计算当前bIdx/n2Idx/s1Idx
             GetActualSeqLen(bIdx, s1Idx); // 获取实际序列长度
             UpdateInnerLoopCond();        // 更新S方向循环条件

             if (curActSeqLenIsZero) {
                 DealActSeqLenIsZero(bIdx, n2Idx);
                 continue;
           }

             // ========== 7. S2方向循环 ==========

             uint32_t curS2LoopTimes = (bn2gIdx == bn2LoopTimes - 1) ?
                 sInnerLoopTimes + 2 : sInnerLoopTimes;

             for (uint32_t sInnerLoopIdx = 0; sInnerLoopIdx < curS2LoopTimes; sInnerLoopIdx++) {

              // ========== 7.1 任务派发阶段 ==========

                 if (sInnerLoopIdx < sInnerLoopTimes) {
                     TaskContext &ctx = taskContext[tasks++];
                     ctx.bidx = bIdx;
                     ctx.nidx = n2Idx;
                     ctx.s2idx = sInnerLoopIdx;
                  // ... 其他参数

                     bool isLast = (bn2gIdx == bn2LoopTimes - 1) &&
                         (sInnerLoopIdx >= sInnerLoopTimes - 1);
```

```
                    if (tasks < TASK_NUM && !isLast) {
                        continue;  // 等待凑够TASK_NUM个任务
                }

                // 派发任务
                    for (uint32_t i = 0; i < tasks; i++) {
                        uint32_t loop = gtasks;
                        CalcParams(loop, extraInfo[loop % EXTRA_NUM], taskContext[i]);

                        if ASCEND_IS_AIV {
                            if constexpr (ANTIQUANT) {
                                QueryPreProcessL(extraInfo[loop % EXTRA_NUM]);
                                CrossCoreSetFlag<SYNC_MODE2, PIPE_MTE3>(SYNC_V0_C1_FLAG);
                           }
                        }
                        gtasks++;
                    }
                }

                // ========== 7.2 C1-V1阶段（MM1+Softmax） ==========

                for (uint32_t i = 0; i < TASK_NUM; i++) {
                    uint32_t loop = gtasksC1V1;
                    if (gloop > 0 && gtasksC1V1 < gtasks) {
                        if ASCEND_IS_AIC {
                            if constexpr (ANTIQUANT) {
                                CrossCoreWaitFlag(SYNC_V0_C1_FLAG);
                           }
                            ComputeMm1(extraInfo[loop % EXTRA_NUM]);
                            CrossCoreSetFlag<SYNC_MODE2, PIPE_FIX>(SYNC_C1_V1_FLAG);
                        }
                        if ASCEND_IS_AIV {
                            CrossCoreWaitFlag(SYNC_C1_V1_FLAG);
                            ProcessVec1L(extraInfo[loop % EXTRA_NUM]);
                            CrossCoreSetFlag<SYNC_MODE2, PIPE_MTE3>(SYNC_V1_C2_FLAG);
                        }
                        gtasksC1V1++;
                    }
                }

                // ========== 7.3 C2-V2阶段（MM2+输出） ==========

                for (uint32_t i = 0; i < TASK_NUM; i++) {
                    uint32_t loop = gtasksC2V2;
                    if (gloop > 1 && gtasksC2V2 < gtasks) {
                        if ASCEND_IS_AIC {
                            CrossCoreWaitFlag(SYNC_V1_C2_FLAG);
                            ComputeMm2(extraInfo[loop % EXTRA_NUM]);
                            CrossCoreSetFlag<SYNC_MODE2, PIPE_FIX>(SYNC_C2_V2_FLAG);
                        }
                        if ASCEND_IS_AIV {
                            CrossCoreWaitFlag(SYNC_C2_V2_FLAG);
                            ProcessVec2L(extraInfo[loop % EXTRA_NUM]);
                        }
                        gtasksC2V2++;
                    }
                }

                tasks = 0;
                gloop++;
            }
        }

        // ========= 8. 等待所有Flags完成 =========

         if ASCEND_IS_AIC {
             WaitFlag<HardEvent::MTE1_MTE2>(L1Q_EVENT0);
            // ... 其他Flags
        }
    }

    // ========== 9. Flash Decode多核同步 ==========

    if constexpr (FLASH_DECODE) {
        SyncAll();  // 全核同步
        if ASCEND_IS_AIV {
            FlashDecodeCompute();
        }
    }
}
```

4.2 ComputeMm1 函数详解

代码位置: `incre_flash_attention_preload_dd.h:3065-3299`

```
template <typename IFAT>
__aicore__ inline void IncreFlashAttentionAttenPreloadDD<IFAT>::ComputeMm1(ExtraInfo& info)
{
    // ========== MM1: Q × K^T 计算 =========

   // 1. 获取Q矩阵L1 Buffer
    LocalTensor<KV_T> aL1Tensor = qL1Buffers[qL1BufIter * L1Q_BLOCK_SIZE / sizeof(KV_T)];

   // 2. 首次循环: 从GM拷贝Q到L1
    if (info.s2Idx == 0) {
        WaitFlag<HardEvent::MTE1_MTE2>(L1Q_EVENT0 + qL1BufIter);
        CopyInMm1AToL1(aL1Tensor, info);  // GM -> L1
        SetFlag<HardEvent::MTE2_MTE1>(L1Q_EVENT0 + qL1BufIter);
        WaitFlag<HardEvent::MTE2_MTE1>(L1Q_EVENT0 + qL1BufIter);
   }

    // ========== 3. K矩阵N方向切分 =========

    constexpr uint32_t nCopyRowCount = KV_LOAD_TO_L1_ROW_NUM;  // = 512
    uint32_t nCopyTimes = (info.actualSingleProcessSInnerSize + nCopyRowCount - 1) / nCopyRowCount;
    uint32_t nTailCopyRowCount = info.actualSingleProcessSInnerSize - (nCopyTimes - 1) * nCopyRowCount;

    // ========== 4. N方向循环 =========

    for (uint32_t nCopyIdx = 0; nCopyIdx < nCopyTimes; nCopyIdx++) {
       // 获取K矩阵L1 Buffer（3槽轮转）
        kpL1BufIter++;
        LocalTensor<KV_T> bL1Tensor = kpL1Buffers[(kpL1BufIter % 3) * L1KP_BLOCK_SIZE / sizeof(KV_T)];
        WaitFlag<HardEvent::MTE1_MTE2>(L1KP_EVENT0 + (kpL1BufIter % 3));

        // Page Attention场景: 三段式搬运K矩阵
        if constexpr (PAGE_ATTENTION) {
            uint64_t blockTableBaseOffset = info.bIdx * maxBlockNumPerBatch;
            uint32_t curSeqIdx = info.s2BatchOffset + nCopyIdx * nCopyRowCount;
            uint32_t copyFinishRowCnt = 0;

            while (copyFinishRowCnt < nActCopyRowCount) {
             // 计算Block Table索引
                uint64_t blockIdOffset = curSeqIdx / kvCacheBlockSize;
                uint64_t reaminRowCnt = curSeqIdx % kvCacheBlockSize;
                uint64_t idInBlockTable = blockTableGm.GetValue(blockTableBaseOffset + blockIdOffset);

             // 计算可拷贝行数
                uint32_t copyRowCnt = kvCacheBlockSize - reaminRowCnt;
                if (copyFinishRowCnt + copyRowCnt > nActCopyRowCount) {
                    copyRowCnt = nActCopyRowCount - copyFinishRowCnt;
             }

             // 计算物理地址
                uint64_t keyOffset = idInBlockTable * kvCacheBlockSize * headDim * kvHeadNum;
             // ... 地址计算

                CopyInMm1BToL1ForPA(bL1Tensor, keyOffset, ...);

                copyFinishRowCnt += copyRowCnt;
                curSeqIdx += copyRowCnt;
          }
         } else {
            CopyInMm1BToL1(bL1Tensor, info, nCopyIdx, nCopyRowCount, ...);
       }

        SetFlag<HardEvent::MTE2_MTE1>(L1KP_EVENT0 + (kpL1BufIter % 3));
        WaitFlag<HardEvent::MTE2_MTE1>(L1KP_EVENT0 + (kpL1BufIter % 3));

        // ========== 5. M方向切分（MSD双段） =========

        constexpr uint32_t baseM = 64 / sizeof(KV_T);
        uint32_t mActCopyRowCount = msdIterNum * info.mSize;  // MSD: 2×mSize
        uint32_t mLoopTimes = (mActCopyRowCount + baseM - 1) / baseM;

        // MSD双段标记
        bool isHead = false;   // 高段（quantScaleC1S1）
        bool isMid = false;    // 中段（跨边界）
        bool isTail = false;   // 低段（quantScaleC1S2）
        bool isOdd = (mLoopTimes % 2) != 0;
        bool hasTail = (mTail == baseM) ? false : true;

        for (uint32_t i = 0; i < mLoopTimes; i++) {
          // 计算当前段类型
            isHead = ...;  // 根据i判断是否是高段
            isTail = ...;  // 根据i判断是否是低段

            // ========== 6. L1 -> L0 搬运 =========

            LocalTensor<KV_T> aL0Tensor = aL0TensorPingPong[(aL0BufIter % 2) * L0A_PP_SIZE / sizeof(KV_T)];
            WaitFlag<HardEvent::M_MTE1>(L0A_EVENT0 + (aL0BufIter % 2));

            LoadData2DParams loadData2DParamsForA;
            loadData2DParamsForA.startIndex = 0;
```

```
    loadData2DParamsForA.repeatTimes = (actualBaseM + 15) / 16 * headDimAlign / (32 / sizeof(KV_T));
    loadData2DParamsForA.srcStride = 1;
    loadData2DParamsForA.dstGap = 0;
    loadData2DParamsForA.ifTranspose = false;
    LoadData(aL0Tensor, aL1Tensor[i * baseM * headDimAlign], loadData2DParamsForA);

    SetFlag<HardEvent::MTE1_M>(L0A_EVENT0 + (aL0BufIter % 2));
    WaitFlag<HardEvent::MTE1_M>(L0A_EVENT0 + (aL0BufIter % 2));

    // ========== 7. B矩阵L1 -> L0搬运（N方向二次切分） ==========

    constexpr uint32_t baseN = 256 / sizeof(KV_T);
    uint32_t nLoopTimes = (nActCopyRowCountAlign + baseN - 1) / baseN;

    for (uint32_t j = 0; j < nLoopTimes; j++) {
        LocalTensor<KV_T> bL0Tensor = bL0TensorPingPong[(bL0BufIter % 2) * L0B_PP_SIZE / sizeof(KV_T)];
        WaitFlag<HardEvent::M_MTE1>(L0B_EVENT0 + (bL0BufIter % 2));

        LoadData2DParams loadData2DParamsForB;
     // ... 参数配置
        LoadData(bL0Tensor, bL1Tensor[...], loadData2DParamsForB);

        SetFlag<HardEvent::MTE1_M>(L0B_EVENT0 + (bL0BufIter % 2));
        WaitFlag<HardEvent::MTE1_M>(L0B_EVENT0 + (bL0BufIter % 2));

        // ========== 8. Mmad矩阵乘 ==========

        MmadParams mmadParams;
        mmadParams.m = actualBaseM;
        if (mmadParams.m == 1) {
            mmadParams.m = 16;  // 避免GEMV模式
     }
        mmadParams.n = actualBaseN;
        mmadParams.k = 128;  // K维度固定128
        mmadParams.cmatrixInitVal = true;  // 初始化C矩阵
        mmadParams.cmatrixSource = false;

        LocalTensor<L0C_T> cL0Tensor = cL0TensorPingPong[(cL0BufIter % 2) * L0C_PP_SIZE / sizeof(L0C_T)];
        WaitFlag<HardEvent::FIX_M>(L0C_EVENT0 + (cL0BufIter % 2));

        Mmad(cL0Tensor, aL0Tensor, bL0Tensor, mmadParams);
        PipeBarrier<PIPE_M>();

        SetFlag<HardEvent::M_FIX>(L0C_EVENT0 + (cL0BufIter % 2));
        WaitFlag<HardEvent::M_FIX>(L0C_EVENT0 + (cL0BufIter % 2));

        // ========== 9. Fixpipe输出（MSD双段处理） ==========

        if (mLoopTimes == 1) {
         // 单次循环：MSD两段分别输出
            for (uint32_t mIter = 0; mIter < msdIterNum; mIter++) {
                float tmp = quantScaleC1S1;
                if (mIter == 1) {
                    tmp = quantScaleC1S2;  // 低段系数
            }

                FixpipeParamsV220 fixParams;
                fixParams.nSize = actualBaseN;
                fixParams.mSize = actualBaseM / msdIterNum;
                fixParams.quantPre = QuantMode_t::DEQF16;
                fixParams.deqScalar = static_cast<uint64_t>(*reinterpret_cast<int32_t *>(&tmp));

                if (mIter == 1) {
                    SetAtomicAdd<half>();  // 第二段原子加
            }

                Fixpipe(mm1ResGm[...], cL0Tensor[...], fixParams);

                if (mIter == 1) {
                    SetAtomicNone();
            }
         }
        } else if (isHead) {
         // 高段：使用quantScaleC1S1
            Fixpipe(mm1ResGm[...], cL0Tensor, fixParams);
        } else if (isTail) {
         // 低段：使用quantScaleC1S2，原子加
            SetAtomicAdd<half>();
            Fixpipe(mm1ResGm[...], cL0Tensor, fixParams);
            SetAtomicNone();
        } else {
         // 中段：跨边界，两段分别输出
         // ... 特殊处理
     }

        cL0BufIter++;
        bL0BufIter++;
}
    aL0BufIter++;
```

```
        }
    }
}
```

五、Ascend C API 详细注释

5.1 数据搬运类 API

DataCopy (GM ↔ L1)

```
/**
 * @brief GM到L1的数据搬运，支持ND到NZ布局转换
 * @param dst 目标LocalTensor（L1）
 * @param src 源GlobalTensor（GM）
 * @param params Nd2NzParams参数结构体
 *
 * Nd2NzParams参数详解：
 * - ndNum: ND矩阵个数
 * - nValue: 单个ND矩阵行数
 * - dValue: 单个ND矩阵列数
 * - srcNdMatrixStride: 相邻ND矩阵偏移（元素）
 * - srcDValue: 同一ND矩阵相邻行偏移（元素）
 * - dstNzC0Stride: NZ矩阵相邻Block偏移（Block数）
 * - dstNzNStride: NZ矩阵相邻行偏移
 * - dstNzMatrixStride: 相邻NZ矩阵偏移
 *
 * 使用示例：
 * Nd2NzParams params;
 * params.ndNum = 1;
 * params.nValue = 512;  // 512行
 * params.dValue = 128;  // 128列（headDim）
 * params.srcDValue = 128;  // GM中相邻行间隔128元素
 * params.dstNzC0Stride = 16;  // NZ布局参数
 * params.dstNzNStride = 1;
 * DataCopy(l1Tensor, gmTensor[offset], params);
 *
 * 注意事项：
 * 1. 源数据必须是ND布局（行主序）
 * 2. 目标数据是NZ布局（16×16 Block列主序）
 * 3. NZ布局与Cube MAC阵列匹配，提升矩阵乘效率
 */
```

LoadData (L1 → L0)

```
/**
 * @brief L1到L0的数据搬运，为Mmad准备数据
 * @param dst 目标LocalTensor（L0A或L0B）
 * @param src 源LocalTensor（L1）
 * @param params LoadData2DParams参数结构体
 *
 * LoadData2DParams参数详解：
 * - startIndex: 起始索引（默认0）
 * - repeatTimes: 重复次数，决定搬运数据量
 * - srcStride: 源步长（默认1）
 * - dstGap: 目标间隙（默认0）
 * - ifTranspose: 是否转置（默认false）
 *
 * repeatTimes计算公式：
 * repeatTimes = (M/16) × (K/blockSize)
 * M维度以16为单位，K维度以blockSize为单位
 *
 * 使用示例：
 * LoadData2DParams params;
 * params.repeatTimes = (actualBaseM + 15) / 16 * headDimAlign / (32 / sizeof(KV_T));
 * LoadData(l0Tensor, l1Tensor[offset], params);
 *
 * 注意事项：
 * 1. repeatTimes最大值255，超过需分批
 * 2. L0A/L0B容量有限（32KB），需合理规划
 * 3. 必须与SetFlag/WaitFlag配合使用
 */
```

DataCopyPad (UB ↔ GM)

```
/**
 * @brief UB与GM数据搬运，自动处理非对齐数据
 * @param dst 目标Tensor
 * @param src 源Tensor
 * @param copyParams DataCopyExtParams参数
 * @param padParams DataCopyPadExtParams参数（可选）
 *
 * DataCopyExtParams参数详解：
 * - blockCount: 行数（UB侧单位为32字节块）
 * - blockLen: 有效数据长度（字节）
 * - srcStride: 源stride（UB侧单位32字节，GM侧单位字节）
```

```
 * - dstStride: 目标stride
 *
 * 使用示例:
 * DataCopyExtParams copyParams;
 * copyParams.blockCount = dealRowCount;
 * copyParams.blockLen = actualColumnCount * sizeof(T);
 * copyParams.srcStride = (columnCount - actualColumnCount) / (BYTE_BLOCK / sizeof(T));
 * copyParams.dstStride = 0;
 * DataCopyPad(gmTensor[offset], ubTensor, copyParams);
 *
 * 注意事项:
 * 1. blockLen非对齐时自动填充dummy数据
 * 2. CopyIn: dummy填充为第一个元素值（isPad=false）
 * 3. CopyOut: 自动丢弃dummy数据
 * 4. 推荐: 所有GM↔UB搬运统一使用DataCopyPad
 */
```

5.2 矩阵计算类 API

Mmad (矩阵乘)

```
/**
 * @brief Cube矩阵乘加速器，执行A×B矩阵乘
 * @param dst 目标LocalTensor（L0C）
 * @param srcA 源A矩阵（L0A）
 * @param srcB 源B矩阵（L0B）
 * @param params MmadParams参数结构体
 *
 * MmadParams参数详解:
 * - m: A矩阵行数（M维度）
 * - n: B矩阵行数（N维度）
 * - k: A/B矩阵公共维度（K维度，固定128）
 * - cmatrixInitVal: 是否初始化C矩阵为0
 * - cmatrixSource: C矩阵来源（false表示新计算）
 *
 * 硬件特性:
 * - MAC阵列: 16×16脉动阵列
 * - 单次MAC：256次乘累加
 * - K维度固定128（INT8）或64（INT16）
 *
 * 使用示例:
 * MmadParams params;
 * params.m = actualBaseM;  // M维度
 * params.n = actualBaseN;  // N维度
 * params.k = 128;          // K维度固定
 * params.cmatrixInitVal = true;
 * Mmad(cL0Tensor, aL0Tensor, bL0Tensor, params);
 *
 * 注意事项:
 * 1. M=1时会触发GEMV模式，不可关闭，需规避（设M=16）
 * 2. 必须与PipeBarrier<PIPE_M>配合使用
 * 3. 结果存储在L0C，需通过Fixpipe输出
 */
```

Fixpipe (L0C → GM)

```
/**
 * @brief L0C结果输出到GM，支持量化/反量化
 * @param dst 目标GlobalTensor（GM）
 * @param src 源LocalTensor（L0C）
 * @param params FixpipeParamsV220参数结构体
 *
 * FixpipeParamsV220参数详解:
 * - nSize: N维度大小
 * - mSize: M维度大小
 * - srcStride: L0C源stride
 * - dstStride: GM目标stride
 * - ndNum: ND矩阵个数
 * - quantPre: 量化模式（DEQF16等）
 * - deqScalar: 反量化系数
 *
 * MSD双段使用:
 * // 高段
 * fixParams.deqScalar = quantScaleC1S1;  // 1/1024
 * Fixpipe(gm[...], cL0, fixParams);
 *
 * // 低段（原子加）
 * SetAtomicAdd<half>();
 * fixParams.deqScalar = quantScaleC1S2;  // 1/(1024×254)
 * Fixpipe(gm[...], cL0[offset], fixParams);
 * SetAtomicNone();
 *
 * 注意事项:
 * 1. 支持原子加，用于MSD双段合并
 * 2. deqScalar需用reinterpret_cast转换
 * 3. 必须与SetFlag/WaitFlag配合
 */
```

5.3 同步类 API

SetFlag / WaitFlag

```
/**
 * @brief 硬件事件同步机制
 * @tparam event 硬件事件类型（HardEvent枚举）
 * @param eventId 事件ID
 *
 * 常用事件类型:
 * - MTE1_MTE2: L1搬运完成同步
 * - MTE2_MTE1: GM到L1搬运完成
 * - M_MTE1: 计算到搬运同步
 * - MTE1_M: 搬运到计算同步
 * - M_FIX: 计算到输出同步
 * - FIX_M: 输出完成同步
 *
 * 使用模式:
 * SetFlag<HardEvent::MTE1_M>(eventId);  // 设置Flag
 * WaitFlag<HardEvent::MTE1_M>(eventId); // 等待Flag完成
 *
 * 注意事项:
 * 1. SetFlag后必须WaitFlag才能使用数据
 * 2. eventId需预先定义（EVENT_ID0-7）
 * 3. 多Buffer轮转需要多个eventId
 */
```

CrossCoreSetFlag / CrossCoreWaitFlag

```
/**
 * @brief 跨Core同步机制，Vector和Cube握手
 * @tparam mode 同步模式（SYNC_MODE2推荐）
 * @tparam pipe 流水线类型（PIPE_MTE3/PIPE_FIX等）
 * @param flagId 同步Flag ID
 *
 * SYNC_MODE2双重语义:
 * 1. pipe_flush: 保证当前引擎数据写回完成
 * 2. 跨Core通知: 通知对方引擎可以开始消费
 *
 * 四级流水握手链:
 * SYNC_V0_C1_FLAG: Vector完成Q预处理 -> Cube开始MM1
 * SYNC_C1_V1_FLAG: Cube完成MM1 -> Vector开始Softmax
 * SYNC_V1_C2_FLAG: Vector完成Softmax -> Cube开始MM2
 * SYNC_C2_V2_FLAG: Cube完成MM2 -> Vector开始输出
 *
 * 使用示例:
 * // Vector侧
 * CrossCoreSetFlag<SYNC_MODE2, PIPE_MTE3>(SYNC_V0_C1_FLAG);
 *
 * // Cube侧
 * CrossCoreWaitFlag(SYNC_V0_C1_FLAG);
 * ComputeMm1(...);
 * CrossCoreSetFlag<SYNC_MODE2, PIPE_FIX>(SYNC_C1_V1_FLAG);
 *
 * 注意事项:
 * 1. 必须Vector先Set，Cube再Wait
 * 2. SYNC_MODE2比SYNC_MODE1多pipe_flush
 * 3. 延迟仅几十cycle，远低于Barrier
 */
```

5.4 算术运算类 API

Cast (精度转换)

```
/**
 * @brief 数据类型转换
 * @param dst 目标LocalTensor
 * @param src 源LocalTensor
 * @param roundMode 舍入模式
 * @param count 元素个数
 *
 * 舍入模式:
 * - CAST_ROUND: 四舍五入（有偏差）
 * - CAST_RINT: IEEE最近偶数舍入（无偏）
 * - CAST_NONE: 不舍入（截断）
 *
 * 使用示例:
 * // FP32 -> FP16（推荐RINT）
 * Cast(fp16Tensor, fp32Tensor, AscendC::RoundMode::CAST_RINT, count);
 *
 * // FP16 -> FP32
 * Cast(fp32Tensor, fp16Tensor, AscendC::RoundMode::CAST_NONE, count);
 *
 * 注意事项:
 * 1. BF16输出必须用CAST_RINT（多轮累加）
```

```
 * 2. 普通FP16可用CAST_ROUND
 * 3. 多轮累加场景CAST_ROUND有正向偏差累积
 */
```

Adds / Muls (标量运算)

```
/**
 * @brief 标量加法（Adds）和标量乘法（Muls）
 * @param dst 目标LocalTensor
 * @param src 源LocalTensor
 * @param scalar 标量值
 * @param count 元素个数
 *
 * 优势:
 * - 单条指令完成，无需Duplicate广播
 * - 节省UB空间（只需32B而非完整广播buffer）
 * - 性能提升约33%
 *
 * 使用示例:
 * // 减法转加法: x - maxVal
 * Adds(dst, src, -maxVal, count);
 *
 * // 除法转乘法: x / sumVal
 * T invSum = (T)1.0 / sumVal;  // CPU端计算
 * Muls(dst, src, invSum, count);
 *
 * 注意事项:
 * 1. 适用于Softmax/LayerNorm逐行处理
 * 2. 除法必须转换为乘法（先计算1/scalar）
 * 3. count传有效数据个数（非对齐数）
 */
```

5.5 归约类 API

ReduceMax / ReduceSum

```
/**
 * @brief 归约最大值和求和
 * @param dst 输出LocalTensor（1个元素）
 * @param src 输入LocalTensor
 * @param tmpBuffer 临时buffer（类型必须与dst相同）
 * @param count 有效数据个数
 * @param calIndex 是否计算索引（ReduceMax可选）
 *
 * 使用示例:
 * LocalTensor<T> reduceTmp = reduceBuf.Get<T>();
 * ReduceMax<T>(maxTensor, srcTensor, reduceTmp, count, false);
 * ReduceSum<T, true>(sumTensor, srcTensor, reduceTmp, count);
 *
 * 注意事项:
 * 1. tmpBuffer类型必须与T相同（不能用uint8_t）
 * 2. count只传有效数据个数（非对齐数）
 * 3. Level 2接口无对齐要求
 */
```

5.6 Softmax类 API

SoftmaxFlashV2

```
/**
 * @brief Flash Attention V2 Softmax实现
 * @param dst 输出Tensor（归一化结果）
 * @param sum 输出Tensor（sum值）
 * @param max 输出Tensor（max值）
 * @param src 输入Tensor（注意力分数）
 * @param exp 临时Tensor（exp值）
 * @param inSum 输入sum（上一轮）
 * @param inMax 输入max（上一轮）
 * @param tmp 临时buffer
 * @param tiling Tiling参数
 * @param shape 形状参数
 *
 * Online Safe Softmax递推公式:
 * m_new = max(m_old, rowmax(S_cur))
 * P_cur = exp(S_cur - m_new)
 * O = O_old × exp(m_old - m_new) + P_cur × V_cur
 * sum_new = sum_old × exp(m_old - m_new) + sum(P_cur)
 *
 * 使用示例:
 * SoftMaxShapeInfo srcShape{dealRowCount, columnCount, dealRowCount, actualColumnCount};
 * SoftMaxTiling tiling = SoftMaxFlashV2TilingFunc(srcShape, sizeof(T), ...);
 *
 * SoftmaxFlashV2<T, true, true, false, false, CFG>(dst, sum, max, src, exp, inSum, inMax, tmp, tiling, srcShape);
 *
```

```
 * 注意事项:
 * 1. 支持跨轮累加 (inSum/inMax)
 * 2. 数值稳定: exp(m_old-m_new)≤1不会溢出
 * 3. 需配合预加载轮转使用
 */
```

六、Page Attention 三段式搬运详解

6.1 三段式原理

代码位置: `incre_flash_attention_preload_dd.h:3405-3524`

```
/**
 * Page Attention的KV Cache按Block分散存储，一次请求可能跨越多个Block。
 * 三段式搬运处理非连续地址:
 *
 * 1. Head段: 首个Block的剩余部分
 *    - copyStartRowCnt % blockK 可能不为0
 *   - 需要特殊偏移计算
 *
 * 2. Mid段: 完整Block逐个搬运
 *   - 每次搬运完整的blockK行
 *   - 通过Block Table获取物理Block ID
 *
 * 3. Tail段: 最后不完整Block
 *   - 剩余行数可能小于blockK
 *
 * Block Table结构:
 * blockTableGm[blockIdx] = physicalBlockId
 * physicalBlockId -> 物理Block起始地址
 *
 * 计算公式:
 * keyOffset = physicalBlockId × kvCacheBlockSize × headDim × kvHeadNum
 *           + n2Idx × headDim  (BNSD布局)
 *           + remainRowCnt × headDim × kvHeadNum (BSH布局)
 */
```

七、性能优化要点总结

优化措施	API/机制	性能提升
预加载流水	PRE_LOAD_NUM_DD=4	隐藏搬运延迟
MSD双段	Fixpipe原子加	精度提升+零开销
Brcb节省UB	Brcb广播	节省~18KB
CrossCore同步	SYNC_MODE2	延迟~几十cycle
L2Cache关闭	SetL2CacheHint	带宽提升5-10%
NZ布局转换	Nd2NzParams	矩阵乘效率+30-50%

本文档基于 *`incre_flash_attention_preload_dd.h`* 代码逐行解析，结合*Ascend 910B/C*硬件特性进行深度技术分析。